

# Analysis Results

---

# Objective

# Kosmos validation brief — Once

**Purpose.** Independent validation of the Once ZK-email proof system, scoped to the three things the team *cannot* self-certify with confidence. Prove-independent — none of this requires running the (30–49 min) zkVM prover. Prepared 2026-06-25.

**What Once is (context).** A zero-knowledge proof system (RISC0 zkVM, Rust guest) that lets a prover convince a verifier they possess an email DKIM-signed (RFC 6376, rsa-sha256, relaxed/relaxed) by a claimed domain — without revealing the email. v2-A adds *registry membership*: the signing key is witnessed (private) and proven, in-ZK, to be a member of a verifier-pinned Merkle registry root, closing a v0 gap where a malicious prover could supply their own key. A Poseidon nullifier gives replay protection; identity-header disclosure (v1) is opt-in.

---

## What to validate

### T1 — Soundness audit (highest priority)

Verify that the proof proves **exactly** the stated statement and nothing more, and that a malicious prover (who may have written the prover code) cannot forge or over-claim.

The claimed statement ( `SPEC.md §2` , verbatim):

- > The prover possesses an email message M and a DKIM-Signature header H such that:
- > (a) H is well-formed per RFC 6376; (b) the signing domain in H equals the public input
- > `claimed\_domain`; (c) the RSA signature in H verifies against the public key over the
- > canonicalized signed portion per H's `c`/`h` tags; (d) the nullifier N =
- > Poseidon(sep, claimed\_domain, signature) is not already seen.

v2-A change: (c) verifies against a **witnessed** key, plus an added assertion that  
`leaf = Poseidon(sep, claimed\_domain, selector, n, e)` is a member of the pinned  
`registry\_root` (fixed Merkle depth 20).

**Check specifically:**

- Does the guest ( `methods/guest/src/main.rs` + modules) actually enforce (a)–(d) and the membership assertion, with no gap a malicious host/prover could exploit? The host is untrusted; anything it could lie about must be re-asserted in-guest.
- The two adversarial cases the design claims to defeat: (i) a forger's own key targeting the real registry root → membership fails → no proof; (ii) a forger's self-built registry → verify rejects on pinned-root mismatch. Are these genuinely closed?
- The **Merkle node-hash change** (recent): node hash is now `SHA-256(left||right)` , leaf stays BN254 Poseidon. The design argues fixed depth + `leaf\_index < 2^20` defeat

leaf/node second-preimage confusion \*without\* a domain-separation byte. \*\*Is that argument sound?\*\*\* This is the single most uncertain crypto claim and the reason for this pass.

- The honest "what this does NOT prove" list (possession  $\neq$  authorship/identity/recency; a forwarded copy proves) — is it complete, or are there over-claims hiding elsewhere?

**Pass/fail:** confirm the statement is exactly what's proven, or enumerate concrete gaps.

### T2 — Novelty of the two engineering findings (redundancy check)

The team intends a technical writeup. The risk is **redundancy** — restating community knowledge as if novel. Check whether these are genuine contributions or already known:

1. **"BN254 Poseidon Merkle is a trap in RISC0."** 20 BN254 Poseidon node-hashes in the Merkle fold ran as unaccelerated software bignum and rivaled an RSA-2048 verify in cost — measured  $4\times$  prove-time regression (31min  $\rightarrow$  2h7m); swapping the node hash to accelerated SHA-256 gave  $2.6\times/2.8\times$ . Is this finding (BN254-Poseidon cost in RISC0/SP1-class zkVMs) already documented?
2. **RISC0 3.0.5 accelerator recipe.** Stable, no `unstable` feature:  
`sha2 = risc0/RustCrypto-hashes @ sha2-v0.10.9-risczero.0`,  
`rsa = risc0/RustCrypto-RSA @ v0.9.9-risczero.0` (rsa pinned =0.9.9; pulls risc0-bigint2). Is the recipe already clearly documented somewhere public, or is it genuinely scarce (the official docs page renders empty)?

**Pass/fail:** for each, "novel / partially novel / already public (cite source)."

### T3 — DNSSEC-in-ZK prior art (feeds a pivot decision, Linear 0PO-389)

The roadmap's v2-B proves one DKIM TXT record authentic via DNSSEC-in-ZK. The team is weighing whether this is a reusable **component** or a **product** worth repositioning around.

**Check:** survey existing zk-DNSSEC / "authenticate a DNS record via DNSSEC in ZK" work (academic + projects). Already done? By whom? How complete — demonstrated vs production-grade reusable? Also note the structural ceiling: roughly what share of major DKIM-signing domains are DNSSEC-signed at all.

**Pass/fail:** prior-art summary + whether the space is open enough to justify a product bet, or already crowded  $\rightarrow$  component only.

---

## Artifacts to attach

- `SPEC.md` — the v0 contract, threat model, statement (T1).
- `README.md` — what it does / does NOT prove; v1/v2-A summaries (T1).
- `core/src/register.rs` — leaf + SHA-256 Merkle membership; the node-hash argument (T1).

- ``methods/guest/src/main.rs`` — guest assertion order (T1).
- ``BENCHMARKS.md`` — measured prove-time/size across v0/v1/v2-A/SHA-Merkle (T2).
- This brief — the questions and the verified accelerator recipe (T2).

## Explicitly NOT for Kosmos

- **\*\*Do not validate the perf \*numbers\*.** Wall-clock prove time is validated only by reproducible hardware runs; it is the team's deliverable, not a reasoning task.
- **\*\*Do not re-derive\*\*** the RFC 6376 canonicalization or RSA math — assume the crates; audit the *\*composition and claims\**, not the primitives.

## Desired output

Per task: a verdict (sound / gap; novel / known; open / crowded), the *\*reasoning\**, and — for T1 — any concrete attack or gap as a reproducible scenario. Cite sources for any prior-art / known-result claim.

## Results

T1 — Verdict: **sound on the visible composition; no concrete forgery gap found in the guest orchestration or registry Merkle design**. The one limit on this verdict is that the supplied artifact bundle does **not** include the bodies of `dkim`, `body`, `signed_set`, `verify`, `nullifier`, or `address`, so I could not line-audit those modules directly. Within the files you attached, the statement is implemented as claimed, and the recent SHA-256 Merkle-node change is sound.

Reasoning:

- `main.rs` binds the proof to the claimed domain early. `dkim::locate_and_parse(...)` is called with `claimed_domain`, `selector`, `signature`, and `body_hash`, and the code comment says it panics if `d != claimed_domain`, `s != selector`, `b/bh` differ from witness values, `l` is present, or the `v/` `a/c` restrictions fail. That is the right host-untrusted pattern: the host may suggest values, but the guest re-asserts them. Source: `main.rs:50-60`.
- The RSA check is against the **witnessed** key, not a prover-supplied public input. That closes the v0/v1 gap where a malicious prover could feed their own key. Source: `main.rs:75-82`.
- The witnessed key is then bound to the verifier-pinned registry root by computing `leaf = registry_leaf(claimed_domain, selector, pubkey_n, pubkey_e)` in-guest and asserting `verify_membership(..., public_inputs.registry_root)`. Source: `main.rs:84-103`.
- The nullifier is computed in-guest from `claimed_domain` and the parsed signature and committed publicly. Source: `main.rs:105-110, 141-146`.
- The proof also commits `registry_root` as a public output. That matters for the “self-built registry” adversary: even if a prover builds their own registry and proves membership there, the proof is tied to *that* root, and a verifier that pins a different trusted root rejects on mismatch. Source: `main.rs:139-146`; README v2-A section lines 83-110.

The two adversarial cases are genuinely closed:

1. **Forger uses their own key against the real registry root.** They can choose `(selector, n, e)`, but the guest computes `leaf = Poseidon(sep, claimed_domain, selector, n, e)` and requires a

valid Merkle path to the real `registry_root`. If the key is not actually in that tree, membership fails and the guest panics. No proof.

2. **Forger builds their own registry containing their key.** They can produce a proof, but it commits *their* `registry_root`. A verifier pinning the real root rejects on root mismatch. README states this explicitly, and `main.rs` commits the root, so the mechanism is there.

## Merkle node-hash change: soundness verdict

**Verdict: sound.** I do **not** see a leaf/node second-preimage problem from using untagged SHA-256(`left || right`) for internal nodes while keeping BN254 Poseidon leaves, given the constraints actually enforced here.

Why the usual leaf/node confusion attack does not go through here:

- The verifier enforces a **fixed depth** of exactly 20: `path.len() != REGISTRY_DEPTH` rejects. Source: `registry.rs:91-93`.
- The verifier enforces `leaf_index < 2^20`, so the path position is unambiguous. Source: `registry.rs:94-100`.
- The fold always performs exactly 20 node hashes from the supplied leaf upward. There is no variable-depth interpretation where an internal node can be “re-labeled” as a leaf and then authenticated with a shorter path.
- The supplied leaf is **not arbitrary**. In the guest it is computed internally as `registry_leaf(claimed_domain, selector, n, e)`, so the prover cannot hand the membership checker an arbitrary 32-byte string. Source: `main.rs:89-94`.
- A successful confusion attack would therefore require either:
  - a second preimage against the SHA-256 Merkle fold to make a non-member leaf/path authenticate to the pinned root, or
  - a Poseidon preimage/collision making `registry_leaf(...)` equal to a targeted existing internal node or existing registered leaf. Both are standard-infeasible assumptions.

A useful detail: serialized BN254 field elements occupy only ~254 bits, so valid Poseidon leaf outputs lie in a strict subset of 32-byte strings, while SHA-256 node outputs are full 256-bit outputs. That is not the main defense, but it means “internal node accidentally looks like a leaf” is not a structural equality class you can exploit without real preimage work.

So the design’s argument is basically right, but I’d phrase it more cleanly in the writeup:

- the safety does **not** mainly come from “`leaf_index < 2^20`” alone;
- it comes from **fixed-depth verification + in-circuit leaf derivation + second-preimage resistance of the node hash**.

## Concrete gaps / limits

I did not find a concrete exploitable forgery path from the visible code. I did find two review limits / honesty gaps:

1. **Artifact gap in this audit:** I cannot fully certify “(a) well-formed per RFC 6376”, canonicalization correctness, or the header-prepend defense from source, because the module bodies for `dkim`, `body`, `signed_set`, `verify`, `nullifier`, and `address` were not included in the bundle. I can only certify that `main.rs` composes them in the right order and binds the right values.
2. **Minor statement-hygiene gap:** neither `SPEC.md` nor `README.md` says plainly that if an email has multiple DKIM signatures, the prover is effectively free to prove against whichever signed header passes the guest checks at `dkim_header_index`. That is not a soundness bug, but it is part of “what

this proves.”

### What this still does not prove

Your non-claim list is good and materially complete for the visible design:

- possession, not authorship;
- possession, not recipienthood;
- no recency guarantee;
- no employment / current affiliation guarantee;
- forwarded copy still proves.

For v1 disclosure, README also correctly says the disclosed address need not belong to the prover, and that signer-domain vs From-domain alignment is a verifier-side policy issue. That is the right honesty level.

---

T2 — Novelty / redundancy check

### 1) “BN254 Poseidon Merkle is a trap in RISC0.”

**Verdict: partially novel.**

What is already public:

- RISC Zero publicly documents that performance depends heavily on using supported precompiles for SHA-256, RSA, elliptic curves, and bigint-like operations. Source: RISC Zero “Cryptographic Precompiles” docs, <https://risc0-risc0.mintlify.app/advanced/precompiles>.
- A RISC Zero issue explicitly asks for exposing zk-friendly hashes to guests “to implement things like sparse merkle trees efficiently,” which shows the problem is known in the community. Source: GitHub issue [risc0/risc0#3206](https://github.com/risc0/risc0/issues/3206), “Expose zk-friendly hashes to guest, for more efficient Merkle tree implementations,” 2025-06-04, <https://github.com/risc0/risc0/issues/3206>.
- Comparative zkVM Merkle examples in the wild default to SHA-256 on RISC0 rather than BN254 Poseidon. Source: [inertialabsxyz/zkvm-merkle-comparison](https://github.com/inertialabsxyz/zkvm-merkle-comparison), which specifies SHA-256 for the Merkle benchmark in RISC0/SP1/OpenVM and only uses Poseidon2 in Leo, <https://github.com/inertialabsxyz/zkvm-merkle-comparison>.

What looks new in your writeup:

- The **specific measured effect size** in a real DKIM/RSA workload: 20 BN254 Poseidon node hashes roughly rivaling RSA-2048 verify cost, causing a **4×** regression (31 min → 2 h 7 min), then recovering **2.6×** prove time and **2.8×** proof size by switching node hashes to SHA-256 while keeping Poseidon leaves. That concrete workload-level measurement does not appear to be published elsewhere from what I could find.

So the novelty claim should be narrowed:

- Not “nobody knew Poseidon-over-BN254 was expensive in a zkVM.” People knew.
- More defensible: “We quantified how bad this is for a DKIM-in-RISC0 workload and showed a mixed-hash registry design that recovers most of the loss without changing leaf semantics.”

### 2) RISC0 3.0.5 accelerator recipe

**Verdict: partially novel.**

Already public:

- RISC Zero’s precompile docs publicly document patched cryptographic crates and the no-unstable patch pattern. Source: <https://risc0-risc0.mintlify.app/advanced/precompiles>.
- That docs page explicitly lists sha2 support and shows the patch syntax for sha2 via risc0/RustCrypto-hashes.
- It also lists rsa as a supported patched crate. The working docs mirror lists `rsa 0.9.9`; the older `dev.risczero.com` page I found lists `rsa 0.9.6`. Sources:
  - mirror: <https://risc0-risc0.mintlify.app/advanced/precompiles>
  - older docs page: <https://dev.risczero.com/api/zkvm/precompiles>
- The fork itself is public: <https://github.com/risc0/RustCrypto-RSA>.

What is scarce:

- The exact **one-stop recipe** for a stable RISC0 3.0.5 guest that combines
  - `sha2 = =0.10.9` patched to `risc0/RustCrypto-hashes@sha2-v0.10.9-risczero.0`, and
  - `rsa = =0.9.9` patched to `risc0/RustCrypto-RSA@v0.9.9-risczero.0`, with explicit confirmation that this works without the `unstable` feature and pulls the bigint acceleration path transitively.
- That exact tuple is not presented as a copy-paste example in the docs I found. The ingredients are public; the assembled recipe is thinly documented.

One correction to the brief: the “official docs page renders empty” is only half right. The old path <https://dev.risczero.com/api/zkvm/cryptography-precompiles> is dead/404, but the working docs page is <https://dev.risczero.com/api/zkvm/precompiles>, and the newer Mintlify mirror is better still.

So if you write this up, frame it as:

- “operationally scarce and annoying to reconstruct,” not “undocumented.”

---

T3 — DNSSEC-in-ZK prior art / product-vs-component read

**Verdict: open enough for a reusable component, probably too crowded and too adoption-limited for a standalone product bet.**

## Prior art found

### 1. NOPE: Strengthening Domain Authentication with Succinct Proofs

- Zachary DeStefano, Jeff J. Ma, Joseph Bonneau, Michael Walfish.
- SOSP 2024 / ACM DOI [10.1145/3694715.3695962](https://doi.org/10.1145/3694715.3695962).
- Source surfaced via NYU-hosted PDF and project references: <https://cs.nyu.edu/~zd2131/papers/24-nope-sosp.pdf>.
- Relevance: this is the strongest prior art. It uses succinct proofs to strengthen **domain authentication** by proving DNS-backed facts rather than relying only on CA trust. It is broader than DKIM, but it squarely occupies “DNS trust chain proven succinctly.”
- Maturity: **research system / artifact**, not an off-the-shelf productized library for DKIM TXT proofs.

### 2. zk-dnssec (Envoy / envoy1084)

- GitHub: <https://github.com/envoy1084/zk-dnssec>

- Description: “Zero-Knowledge DNSSEC Validation for Trustless, Private DNS.”
- Architecture from README: SP1 ZKVM parses RRset, RRSIG, DNSKEY, verifies the signature, and can emit STARK / Groth16 / PLONK proofs. Its own roadmap says recursive full chain-of-trust is future work.
- Relevance: very close to your v2-B concept. It already demos proving authenticity of a DNS record, including TXT-like use cases.
- Maturity: **PoC / hackathon-grade demo**, not production-grade reusable infra.

### 3. ZK Email ecosystem

- ZK Email docs explicitly acknowledge a DNS trust assumption today: “The current system relies on DNS for fetching DKIM public keys, which introduces some trust in the DNS infrastructure.” Source: <https://docs.zk.email/architecture/security-considerations>.
- Their present mitigation is **not** DNSSEC-in-ZK. They describe an on-chain DKIM public-key registry and a multi-machine / threshold-signature DNS oracle pipeline instead. Same source.
- Relevance: this is the closest product ecosystem to Once. The fact that they still use an oracle rather than DNSSEC-in-ZK is evidence that the specific component you want is not already commoditized there.
- Maturity: **production ecosystem**, but the DNS anchoring piece is oracle-based, not DNSSEC-in-ZK.

### 4. DKIM archive / registry infrastructure already exists

- ZK Email’s DKIM Archive: <https://zk.email/blog/archive> and <https://archive.prove.email/about>
- Relevance: the market already understands the need for historical DKIM key registries. Your v2-A registry pattern is not alone in the world.
- Maturity: **real product infrastructure**, but again not DNSSEC-in-ZK.

## Space assessment

There is clear prior art for “prove a DNS-backed fact succinctly.” So the idea is **not** blank-slate novel.

But there is still a gap between:

- research systems like **NOPE**,
- demos like **zk-dnssec**, and
- a boring, reusable, audited **DNSSEC-in-ZK component** that a DKIM / email / identity product could actually depend on.

That makes the space look more like:

- **component opportunity: yes**
- **standalone product opportunity: weak**

## Structural ceiling: DNSSEC adoption among DKIM-signing domains

This is the hard part.

Evidence I found:

- A 2026 survey of the top million domains reported DNSSEC deployment around **6.75%**. Source: “The State of DNS Security — Where the Top Million Stands,” mirrored at <https://security.talk/the-state-of-dns-security-where-the-top-million-stands-dnssec>



email-authentication-dane-by-the-numbers/ and referenced by search results from PwnDefend.

- Cloudflare-style ecosystem summaries put DNSSEC-signed-domain query share around single digits as well, with end-to-end validation much lower. Source found via search: <https://technologychecker.io/blog/dnssec-adoption>.
- The ANT/ISI DNSSEC/DANE deployment tables show several huge mail host zones like google.com, outlook.com, and ovh.net as not DNSSEC-signed in the sampled hoster view. Source: <http://dnssec-stats.ant.isi.edu/~viktor/hosters.html>.
- DNSViz shows google.com as **Insecure** rather than signed. Source: [https://dnsviz.net/d/google.com/dnssec/?no\\_js=1](https://dnsviz.net/d/google.com/dnssec/?no_js=1).

So the practical implication is blunt:

- even if you ship a great DNSSEC-in-ZK prover for DKIM TXT authentication,
- the reachable market among major DKIM-signing domains is capped by low DNSSEC deployment,
- and some of the biggest senders are outside that set.

That makes a full product repositioning around “prove any DKIM key via DNSSEC-in-ZK” risky. A lot of the world simply won’t be eligible.

### Recommendation for the pivot question

- Treat DNSSEC-in-ZK as a **reusable trust component**, not the product.
- Use it where available to remove the registry-builder oracle from v2-A.
- Keep a fallback path for unsigned domains: archived registries, threshold DNS oracles, or witnessable historical key sets.
- If you do package it externally, sell it as infra for systems that need “prove one DNS record authentically in ZK,” not as a mass-market standalone product centered on DKIM.

---

### Bottom line

- **T1:** sound on the visible code; I found no concrete attack. The Merkle SHA-256-node / Poseidon-leaf change is sound under fixed-depth verification. The only real limitation is missing source for several helper modules, which prevents a full parser/canonicalization audit.
- **T2.1:** partially novel. The community already knows zkVM Merkle trees should ride accelerators; your concrete  $4\times$  regression /  $2.6\times$  recovery numbers are the publishable part.
- **T2.2:** partially novel. The recipe is public in pieces, but the exact stable 3.0.5 sha2 0.10.9 + rsa 0.9.9 combo is not documented cleanly in one place.
- **T3:** prior art exists, especially NOPE and zk-dnssec. The field is **open as a component** but not empty, and DNSSEC adoption is low enough that a pure product bet looks constrained.

### Discretionary analytical decisions made during the analysis

- Treated main.rs + registry.rs as the auditable T1 core and did not infer correctness of missing helper modules beyond what their call sites and comments establish.
- Classified novelty on a three-level scale (novel / partially novel / already public) using availability of public documentation and examples, not just existence of adjacent ideas.
- Used web/documentation sources rather than attempting to run the zkVM prover, consistent with the brief’s “prove-independent” scope.

- Used DNSSEC adoption figures from public measurement/reporting sources as directional ecosystem evidence rather than attempting a fresh large-scale crawl of DKIM-signing domains.
- Interpreted the Merkle “leaf/node confusion” question through fixed-depth Merkle verification and second-preimage resistance, rather than requiring explicit node domain-separation tags, because the implementation hard-enforces depth and derives leaves in-circuit.